

ECE 2036 Lab 2: "Making Dunder-Mifflin a Better Place"

Assigned: Sept. 2, 2020

Due: Sept. 14, 2020 at 11:59PM

Reading: Deitel & Deitel Chapter 2-4

Video Lectures Covered: Topics 1-3, plus some of 4

You have just been hired by Michael Scott as the new office engineering intern. Corporate has questioned why Dunder-Mifflin needs an engineer, but Michael Scott has made the case that he wants to optimize his office so that it is a well-oiled machine. And, of course, who better than a Georgia Tech undergraduate student who is a master in C++ to be this intern! In an effort to start making his sales force more efficient, you are ordered to report to the Assistant (to the) Regional Manager, Dwight Schrute. Dwight wants to optimize his traveling schedule for each day so that he can take the shortest path for all his daily traveling locations. He has many places in Scranton to visit every day, which include checking up on clients, attend to his beet farm, meet with his volunteer police duties, go to study at his dojo, etc... Optimization is at the top of the list, and as an engineer this is right in your wheelhouse! If this pilot program goes well (i.e. does not result in a complete disaster) then Michael Scott will roll your new program out to all of the sales force. Pam is helping you in this endeavor, and she will be in charge of your system when your internship completes.

As seen below, Dwight has faxed you a copy of a map of Scranton and has roughly marked the 24 places in and around Scranton that he might visit as a part of his daily duties. The exact coordinates for each location, plus three more locations further away from Scranton city center, appear in Appendix A.



Dwight insisted on "clicks" for these grid points!

The learning objectives you told your Georgia Tech advisor, who approved this internship, are the following:

1. Using basic C++ classes;
2. Creating basic arrays of user-defined objects;
3. Creating constructors and overloading constructors;
4. Using and creating setter and getter member functions in C++ classes;
5. Using C++ string objects;
6. Using basic text file I/O objects and operators; and
7. Exploring an optimization problem that can be difficult to solve that is NP-complete.

Software engineering design is about finding problems, having insight, and solving those problems. For this lab you might consider the following ideas to set the stage!

Problem: Dwight Schrute has many places in Scranton that he needs to visit in one day, and he is obsessed with optimizing time management (as evidenced by the jar he has at his desk and in his car). The more optimized he is with his time, the more money that he can make for himself and Dunder-Mifflin.

Insight: After observing his day, you and Pam realize that Dwight seems to randomly pick places to go in Scranton related to his to-do list. Although he won't admit it, he is unconsciously driven by his immediate desires and thoughts. You and Pam notice that he does seem to waste a lot of time simply travelling haphazardly from one place to another (along with some unusual breaks in the warehouse after he puts the bobblehead of himself on his desk).

Solution: You confirm that to help optimize the office, Dwight (and perhaps the rest of the salesforce and the warehouse crew perhaps) needs a software system to optimize the order of the locations that they need to travel to in one day.

User Interface and System Requirements

The user interface is very basic, and it will prompt the user to enter the locations in Scranton that they need to visit in a given day. It is assumed that a Scranton employee, like Dwight Schrute, will always start and end at the Scranton Branch Office. The user, however, does not have to enter the Scranton Branch Office as their first or last location because the software system will insert this automatically. Furthermore, you can assume that the employee will only put in a location ONCE (no error checking needed in this version). They will input a location number (that is defined in Appendix A). Please note that the specific user input is indicated in **bold** below.

Please make your system prompt the user in the following way on the next page.

Good Morning Dwight Schrute!!!

Please list the locations that you need to visit today
(input 0 to end your list)

1 (Scranton Office Branch)

2

12

4

17

6

26

8

13

10

24

22

18

0

Input Requirements

In addition to getting information from the user described above, your program will also need to read in the text file from Appendix A: `ScrantonMapCoordinates.txt`. The X and Y coordinate of each location is needed in order to calculate the distance between locations. PLEASE calculate the distance between two locations using ORTHOGONAL distances only (i.e. the absolute value of the difference in the X coordinates added to the absolute value of the difference in the Y coordinates). The units of these coordinates are in “clicks” or kilometers.

Output Requirements

I would like for you to output various paths that you calculate with different algorithms, and put the output data in a text file called `output.txt`. Please hardcode “`output.txt`” in your source code. I would like for you to compare the paths produced by a simple heuristic algorithm, a greedy algorithm, and a simulated annealing algorithm. The output should be in the format in Appendix C, and I will describe each algorithm in Appendix D.

Programming Requirements

1. Please only use static (i.e. not dynamic) arrays in your classes. You can make them sufficiently large so that it can handle a full list of locations (i.e. 27). Later we will learn about dynamic arrays, but using them in classes at this point can be a little tricky!

2. I will require that you have at least the following classes listed below, which are basically a `MapLocation`, `Map`, and `Schedule` classes. I would like for you to have two source files and

two corresponding header files for your classes. I will provide you a `main.cc` file that appears in Appendix B. Please make sure that your system works with the main file provided to you. You will need to have some inclusion guards in each header file. For example, in your `maps.h` you need to have the following additional preprocessor directives:

```
#ifndef MAPS_H
#define MAPS_H

//your class interfaces here for MapLocation first and then Map

#endif
```

For your `schedule.h` you will need to have the following

```
#include <string>
#include "maps.h"

#ifndef SCHEDULE_H
#define SCHEDULE_H

//your class interface here for a schedule

#endif
```

A. Classes in both `maps.h` (interface) and `maps.cc` (implementations)

MapLocation Class

This class will be called `MapLocation`. You will need to create a class with data members that hold the ID number for each location on the map (perhaps an int variable), the name of the location (a string variable), the x coordinate (a float variable), and the y coordinate (a float variable) for a given location. You can choose your own data member names. Please make sure that you have a full set of setters and getters for this class. Also, please have a default constructor and a constructor with 4 arguments that will allow you to initialize all 4 parameters in this class.

Map Class

This class will be called `Map`. It will contain a static `ARRAY` of `MapLocations` and the number of locations. For simplicity you can define a constant using a preprocessor directive constant that has a fixed number of elements needed for this assignment. Actually, you will only need 28 maximum. We have not talked about `const` designation for variables yet, so I will recommend the following preprocessor directive that defines a constant that you will need to put in your header file.

```
#define NUM_LOCATIONS 28
```

Therefore, when you declare the array in this `Map` class, you can have something like the following:

```
MapLocation locations[NUM_LOCATIONS];
```

Later I will show you how to have a dynamic array to be managed inside a class. I will leave it up to you if you need setters or getters for this class.

void initializeMap(std::string)

This member function, which is called in `main.cc`, will need to read in the input file, `ScrantonMapCoordinates.txt`, that has all the map coordinates and put the values in the `locations` array describe above. I have provided this for you on Canvas.

B. Classes in both `schedule.h` (interface) and `schedule.cc` (implementations)

Schedule Class

This class will be called `Schedule`. You will need to create a class with data members that hold the employee's name, number of locations to visit, and array of `MapLocations` for the list of locations that need to be visited, and a second array of `MapLocations` that has the `routeLocations` array that you create. This `MapLocations` array will be static arrays that are sized to fit the maximum number of elements, and the `int` variable that contains the number of locations to visit will allow you to manage the array appropriately. You can choose your own data member names.

I would suggest that you have a set of "utility functions." I have potentially not said too much about "utility functions," but these are private member functions that can only be used with the class itself. The "client" will not be able to use them. In this case you can make these functions private to `Schedule` that will be used internally in the class member functions to generate the lists needed as specified in the output requirements of the program. For example, you MIGHT have something like the following in the class interface for the `Schedule` class.

private:

```
float calcTotalDistance(MapLocation[]); //Pass either toDoList or routeLocations
void swap(MapLocation[], int, int);
void createInitialRoute();
void greedyOptimization();
void SAOptimization();
float calcDistance(MapLocation, MapLocation);
```

Furthermore, as seen in the main function that I provide for you, you will need to have **PUBLIC** member functions to get the initial input from the user (i.e. `getToDoList`) and generate all the specified routes (i.e. `generateRoute`). Also, please have a default constructor and a constructor with a single string argument that sets the name of the person who the `Schedule` object belongs to (as seen in the main function provided).

Compiling Your Program

To compile and link all the files that I would like for you to manage. You will need to use the following command on coc-ice.pace.gatech.edu. (You can use a makefile as well if you so desire as mentioned in the videos)

g++ maps.cc schedule.cc main.cc -o run_me_dwight_schrute

APPENDIX A: ScrantonMapCoordinates.txt
(Note that grid points are in kilometers!)

1 Dunder_Mifflin_Branch_Office	7.2	5.9
2 Shrute_Beet_Farm	0.5	13.5
3 Barbara_Allen_Business	0.5	7.0
4 Goju_Ryo_Karate_Dojo	1.8	6.1
5 Scranton_Business_Park	0.2	4.3
6 Alfredos_Pizza_Cafe	2.2	3.9
7 Scranton_Chamber_of_Commerce	6.0	6.1
8 Staples	7.4	3.5
9 Pizza_by_Alfredo	4.4	1.0
10 Sheriffs_Office	6.2	2.5
11 Shoe_La_La	1.0	2.1
12 Aaron_Grandy_Attorney_at_Law	2.5	1.5
13 Larry_Meyers_Office	2.9	2.6
14 Scranton_Times_Building	4.0	6.1
15 Blue_Cross_of_Pennsylvania	2.5	5.3
16 Dunmore_High_School	7.1	6.9
17 Harper_Collins	3.8	1.3
18 Prestige_Postal_Company	0.5	8.8
19 US_Post_Office	4.5	3.5
20 White_Pages	6.5	1.2
21 East_Pennsylvania_Seminary	6.9	8.5
22 Serenity_by_Jan	4.0	9.1
23 The_Pet_Emporium	5.1	7.7
24 Tract_Industries	5.5	6.5
25 Apex_Technology	14.6	7.8
26 Haymont_Tires	10.7	4.5
27 Phil_Maguire_Business_Park	12.9	24.3

APPENDIX B: **main.cc**

```
#include "maps.h"
#include "schedule.h"

using namespace std;

int main()

{
    system("clear");
    srand(2020);

    //get input for coordinates and store in an internal array
    //For simplicity you may assume that this is fixed at 24 locations

    Map ScrantonMapCoordinates;
    ScrantonMapCoordinates.initializeMap("ScrantonMapCoordinates.txt");

    //Now instantiate a DwightSchruteSchedule object and get the locations
    //that he needs to visit today. Remember he needs to
    //begin and end at the Scranton Office Branch.

    Schedule DwightSchruteSchedule("Dwight Schrute");
    DwightSchruteSchedule.getToDoList(ScrantonMapCoordinates);
    DwightSchruteSchedule.generateRoute();

}
```


APPENDIX C: example output.txt file

Todo list from worker

Dunder_Mifflin_Branch_Office(1) to Shrute_Beet_Farm(2) = 14.3

Shrute_Beet_Farm(2) to Aaron_Grandy_Attorney_at_Law(12) = 14

Aaron_Grandy_Attorney_at_Law(12) to Goju_Ryo_Karate_Dojo(4) = 5.3

Goju_Ryo_Karate_Dojo(4) to Harper_Collins(17) = 6.8

Harper_Collins(17) to Alfredos_Pizza_Cafe(6) = 4.2

Alfredos_Pizza_Cafe(6) to Haymont_Tires(26) = 9.1

Haymont_Tires(26) to Staples(8) = 4.3

Staples(8) to Larry_Meyers_Office(13) = 5.4

Larry_Meyers_Office(13) to Sheriffs_Office(10) = 3.4

Sheriffs_Office(10) to Tract_Industries(24) = 4.7

Tract_Industries(24) to Serenity_by_Jan(22) = 4.1

Serenity_by_Jan(22) to Prestige_Postal_Company(18) = 3.8

Prestige_Postal_Company(18) to Dunder_Mifflin_Branch_Office(1) = 9.6

TOTAL DISTANCE = 89

Create initial Path with nearest neighbor

Dunder_Mifflin_Branch_Office(1) to Tract_Industries(24) = 2.3

Tract_Industries(24) to Goju_Ryo_Karate_Dojo(4) = 4.1

Goju_Ryo_Karate_Dojo(4) to Alfredos_Pizza_Cafe(6) = 2.6

Alfredos_Pizza_Cafe(6) to Larry_Meyers_Office(13) = 2

Larry_Meyers_Office(13) to Aaron_Grandy_Attorney_at_Law(12) = 1.5

Aaron_Grandy_Attorney_at_Law(12) to Harper_Collins(17) = 1.5

Harper_Collins(17) to Sheriffs_Office(10) = 3.6

Sheriffs_Office(10) to Staples(8) = 2.2

Staples(8) to Haymont_Tires(26) = 4.3

Haymont_Tires(26) to Serenity_by_Jan(22) = 11.3

Serenity_by_Jan(22) to Prestige_Postal_Company(18) = 3.8

Prestige_Postal_Company(18) to Shrute_Beet_Farm(2) = 4.7

Shrute_Beet_Farm(2) to Dunder_Mifflin_Branch_Office(1) = 14.3

TOTAL DISTANCE = 58.2

Greedy algorithm with random swapping

Dunder_Mifflin_Branch_Office(1) to Tract_Industries(24) = 2.3

Tract_Industries(24) to Goju_Ryo_Karate_Dojo(4) = 4.1

Goju_Ryo_Karate_Dojo(4) to Alfredos_Pizza_Cafe(6) = 2.6

Alfredos_Pizza_Cafe(6) to Larry_Meyers_Office(13) = 2

Larry_Meyers_Office(13) to Aaron_Grandy_Attorney_at_Law(12) = 1.5

Aaron_Grandy_Attorney_at_Law(12) to Harper_Collins(17) = 1.5

Harper_Collins(17) to Sheriffs_Office(10) = 3.6

Sheriffs_Office(10) to Staples(8) = 2.2

Staples(8) to Haymont_Tires(26) = 4.3

Haymont_Tires(26) to Prestige_Postal_Company(18) = 14.5

Prestige_Postal_Company(18) to Shrute_Beet_Farm(2) = 4.7

Shrute_Beet_Farm(2) to Serenity_by_Jan(22) = 7.9

Serenity_by_Jan(22) to Dunder_Mifflin_Branch_Office(1) = 6.4

TOTAL DISTANCE = 57.6

Simulated annealing (SA) algorithm with random swapping

Dunder_Mifflin_Branch_Office(1) to Haymont_Tires(26) = 4.9

Haymont_Tires(26) to Staples(8) = 4.3

Staples(8) to Sheriffs_Office(10) = 2.2

Sheriffs_Office(10) to Harper_Collins(17) = 3.6

Harper_Collins(17) to Aaron_Grandy_Attorney_at_Law(12) = 1.5

Aaron_Grandy_Attorney_at_Law(12) to Larry_Meyers_Office(13) = 1.5

Larry_Meyers_Office(13) to Alfredos_Pizza_Cafe(6) = 2

Alfredos_Pizza_Cafe(6) to Goju_Ryo_Karate_Dojo(4) = 2.6

Goju_Ryo_Karate_Dojo(4) to Prestige_Postal_Company(18) = 4

Prestige_Postal_Company(18) to Shrute_Beet_Farm(2) = 4.7

Shrute_Beet_Farm(2) to Serenity_by_Jan(22) = 7.9

Serenity_by_Jan(22) to Tract_Industries(24) = 4.1

Tract_Industries(24) to Dunder_Mifflin_Branch_Office(1) = 2.3

TOTAL DISTANCE = 45.6

APPENDIX D: Additional Optimization Explanations

1. Todo List from Worker

This first section of your output uses the order from the user input as a path. You will need to make sure that you start at Scranton Office Branch and end at Scranton Office Branch on the list. You will output the orthogonal distance between each location, and then calculate and display the total distance travelled in one day. The units are kilometers. This output appears in APPENDIX C.

2. Create Initial Path with Nearest Neighbor Heuristic

For this section, I would like for you to start with the Scranton Office Branch and then find the NEXT location that is the closest. This will be the second place Dwight will travel. From that location, you will again try to find the closest location and go to that location. You will repeat this process in your code until all the locations have been visited. Finally, you can add the Scranton Office Branch to the end of your list.

This is not necessarily an optimized list, but I would like this to be the starting configuration that you will use in the next two optimizations.

3. Greedy Algorithm with Random Swapping

For this optimization, I would like for you to use a "greedy" algorithm to see what effect it has on your optimization. In short, a greedy approach will only allow you to change the cost function such that you are only reducing the cost. The cost function in this case will be the total distance travelled to visit each of the locations just once while starting and ending at the Scranton Office Branch. **Please note that your starting path for the greedy algorithm will be the path that you create with the nearest neighbor heuristic.**

To do this, I would suggest that you randomly pick two locations (NOT THE SCRANTON OFFICE BRANCH LOCATIONS) and swap their positions in your `routeList`. After the swap, compare the total distance travelled to what it was BEFORE the swap. If the swap results in a REDUCTION in the cost function, then accept the swap. If the swap results in an INCREASE in the cost function, then you can consider it a "bad" swap and return the locations to their original position. A greedy algorithm reject all bad swaps! Repeat this for approximately 100 times the number of elements in your list.

Although this might sound like a reasonable approach to find the GLOBAL minimum in the cost function, it turns out that a greedy method that only accepts good swaps will typically get stuck in a LOCAL minimum. The next algorithm, which is called simulated annealing, is a powerful (but simple) technique to overcome this problem and allow you to find a solution that is closer to a global minimum.

4. Simulated Annealing (SA) Algorithm with Random Swapping

I will discuss this algorithm in class, but basically this algorithm is based upon observations in nature when producing crystalline material. A material that is in a crystal formation (i.e. perfect periodicity in all directions) is a material system that is in its lowest energy state. Energy in this analogy would be the "cost function" that we would like to minimize. To produce a material in this state, experimentalists will heat up the material to a high temperature and then slowly cool the material according to what is called an "annealing schedule". This temperature schedule typically needs to be reduced slowly in order to reach a minimum energy state. If you cool it too quickly, the material will be amorphous and far from a crystalline state (and lowest energy).

The analogy with optimization is that you start the system at a high "temperature", which basically means that there is a high probability that you will accept "bad swaps." As you "cool" your optimization, the probability that you will accept "bad swaps" starts to decrease according to a Boltzmann probability distribution which looks something like the following

$$\text{probability of energy change} = e^{\frac{-(\Delta E)}{kT}}$$

This technique is actually quite easy to code. I will not worry about efficiency here, and I will provide you with some code that you can use. I have this as a private function of the `Schedule` class. You may have to adapt it slightly to make it work for your case. I assume that I have a `RouteList` array that is an array of `MapLocations` which is equal in size to the number of items that Dwight puts into his `todoList`. You can play around with the annealing schedule to see if you can find a more optimized solution as well if you would like! ;-)

```

void Schedule::SAOptimization()
{
    float prob;
    float throwDice;
    //this is a linear ramping down of the temperature
    for (float temp = 15.0; temp >=0; temp-=0.1)
    {
        for (int i = 0; i < numItemsToDo*SCALE_FACTOR; i++)
        {
            //randomly pick two elements to swaps between
            // index 1 and numItemsToDo-1
            int pos1 = rand()%(numItemsToDo-1)+1;
            int pos2 = rand()%(numItemsToDo-1)+1;
            //I will allow pos1 and pos2 to be possibly equal for simplicity

            float dist1 = calcTotalDistance(RouteList);

            swap(RouteList, pos1, pos2);

            float dist2 = calcTotalDistance(RouteList);

            dist2 < dist1 ? prob =1.0 : prob = exp(-(dist2 - dist1)/(0.5*temp));

            throwDice = static_cast<float>(rand())/RAND_MAX;

            if (throwDice > prob) //reject the swap .. i.e. swap back
            {
                swap(RouteList,pos1,pos2);
            }
        }
    } //end temperature loop
}

```

APPENDIX E: Turnin Procedures

1. I would like for you to make a directory on coc-ice.pace.gatech.edu called Lab2. To make this directory, type the following while you are in your home directory.

```
mkdir Lab2
```

2. Please go back to your home directory. You can do this by typing `cd` at the command prompt.
3. At the home directory, please type the following command to create a compressed tarball of your work

```
tar -cvzf yourusernameLab2.tar.gz ./Lab2
```

4. As I showed you in class, you will need to download this file to your local machine. You will then submit this `yourusernameLab2.tar.gz` to canvas so that the TAs can grade it.

APPENDIX F: ECE 2036 Lab Grading Rubric

If a student's program runs correctly and produces the desired output, the student has the potential to get a 100 on his or her lab; however, TA's will **randomly** look through this set of "perfect-output" programs to look for other elements of meeting the lab requirements. The table below shows typical deductions that could occur.

In addition, if a student's code does not compile, then they will have an automatic 30% deduction on the lab. Code that compiles but does not match the sample output can incur a deduction from 10% to 30% depending on how poorly the output matches the output specified by the lab. This is in addition to the other deductions listed below or due to the student not attempting the entire assignment.

AUTOMATIC GRADING POINT DEDUCTIONS

Element	Percentage Deduction	Details
Does Not Compile	30%	Program does not compile on pace ice cluster!
Does Not Match Output	10%-30%	The program compiles but doesn't match all output exactly (seed your srand(2020) in main.cc)

ADDITIONAL GRADING POINT DEDUCTIONS FOR RANDOMLY SELECTED PROGRAMS

Element	Percentage Deduction	Details
Correct file structure	10%	Does not use both .cc and .h files, implementing class prototype correctly
Encapsulation	10%	Does not use correct encapsulation in object-oriented objects
Setters/Getters	10%	Does not use setters and getters for each data member.
Constructors	10%	Does not implement constructors with the correct functionality.
Clear Self-Documenting Coding Styles	5%-15%	This can include incorrect indentation, using unclear variable names, unclear comments, or compiling with warnings. (See Appendix D)

LATE POLICY

Element	Percentage Deduction	Details
Late Deduction Function	score - $(20/24)*H$	H = number of hours (ceiling function) passed deadline note : Sat/Sun count has one day; therefore $H = 0.5*H_{weekend}$

Appendix G: Good Programming Practices

Indentation

When using *if/for/while* statements, make sure you indent 2 to 4 spaces for the content inside those. For example...

```
for(int i; i < 10; i++)  
    j = j + i;
```

If you have nested statements, you should use multiple indentions. Your *if/for/while* statement brackets `{ }` can follow two possible conventions. Each both getting their own line (like the *for* loop) OR the open bracket on the same line as the statement (like for the *if/else* statement) and closing bracket its own line. If you have *else* or *else if* statements after your *if* statement, they should be on their own line.

```
for(int i; i < 10; i++)  
{  
    if(i < 5) {  
        counter++;  
        k -= i;  
    }  
    else {  
        k += i;  
    }  
    j += i;  
}
```

Camel Case

This naming convention has the first letter of the variable be lower case, and the first letter in each new word be capitalized (e.g. firstSecondThird). This applies for functions and member functions as well! The main exception to this is class names, where the first letter should also be capitalized.

Variable and Function Names

Your variable and function names should be clear about what that variable or function is. Do not use one letter variables, but use abbreviations when it is appropriate (for example: "imag" instead of "imaginary"). The more descriptive your variable and function names are, the more readable your code will be. This is the idea behind self-documenting code.

Clear Comments

Some good opportunities to use comments are...

- Introducing a member function or class
- Introducing a section of code with long implementation
- Your name, class information, etc. at the beginning of the file

Appendix H: Input and Output Text Files

For this lab, you will do some basic manipulation of text files. In this section, I will show you how to instantiate an input or output text file object. In addition, you can use the insertion stream (<<) and extraction stream (>>) operators to send data to an output file or receive data from an input file, respectively.

Preprocessor Directive

You will need to have the following include statement in your header file that allows you to use the C++ standard library for I/O files.

```
#include <fstream>
```

Instantiating Output File Objects

To instantiate an output file object that you can use to manipulate your output text file, you will need something like the following.

```
std::ofstream YourOutputFileObject("outputfile.dat", std::ios::out);
```

The "outputfile.dat" name is an arbitrary identifier, and this constructor will create a file in the local directory that you execute your program in. The `std::ios::out` is a designation that you are creating an output file; any existing file with the same name will be overwritten.

Instantiating Input File Objects

To instantiate an input file object from which you can read data, you can do the following:

```
std::ifstream YourInputFileObject("inputfile.dat", std::ios::in);
```

Like the output file example, the "inputfile.dat" name is arbitrary and specifies the name of the file in the local directory from which you would like to read. As you see in the main file, you can use this file object with the `!` operator to check to see if the file is valid.

For this lab, you will have the file in a C++ string. Therefore, you might have the beginning part of your `initializeMap` function look like:

```
void Map::initializeMap(string name)  
{  
    ifstream inputMap(name.c_str(), ios::in);  
  
    //you will need more STUFF after this :-)
```

Insertion Stream Operator

Just like with the `cout` object, you can use the `<<` operator to write data to an output file. You use the object name in place of `cout` in the following way.

```
YourOutputFileObject << "Hello File! " << std::endl;
```

Extraction Stream Operator

Furthermore, just like the `cin` object, you can use the `>>` operator to read data from an input file. You can use the object name in the following way.

```
YourInputFileObject >> string1;
```

This command will read a *single* string from the input file that is delineated by white space. For example, if the input file has the following text:

```
hello from professor Snape
```

The above line would have "hello" stored in `string1` with *no spaces*. Furthermore, the operation itself will return a true value if a string is successfully read in from the input file. This can enable you to embed this statement in a while loop condition to access all the strings in a file sequentially. For example, the following while loop will continue until all the strings have been read into the program one at a time.

```
while (YourInputFileObject >> string1)
{
    //manipulate value in string1
}
```

For this lab you will have to input various items from the input file provided to you in Appendix A. Therefore, you might have something like the following to pick out each piece of data.

```
while (inputMap >> tempID >> tempString >> tempX >> tempY)
{
    //you do something here
}
```